Docker & Docker Compose

Lecture 2

greenwich.edu.vn

UNIVERSITY OF GREENWICH

Alliance with FPT Education

Lesson's Agenda

- Section 1: The evolution of deployment
- Section 2: Docker Core Concepts
- Section 3: Docker Deep Dive
- Section 4: CLI Commands & Networking
- Section 5: Docker Compose

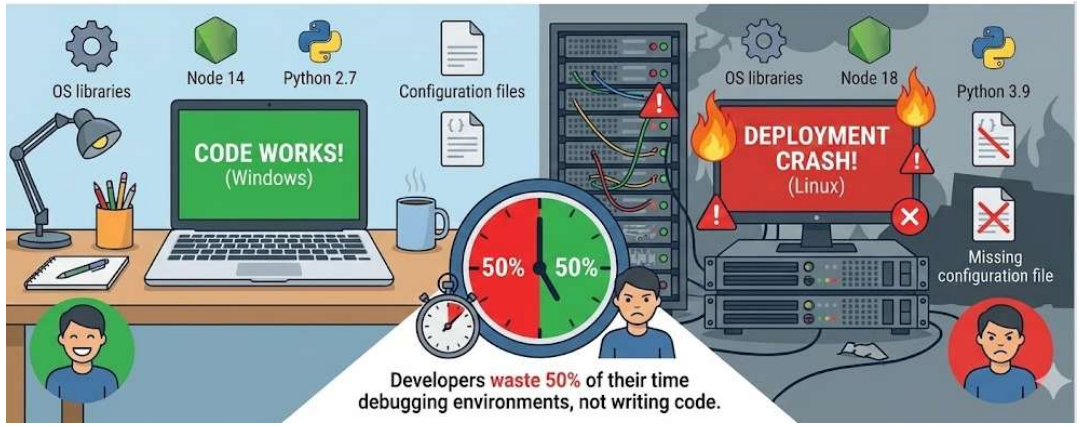
UNIVERSITY OF GREENWICH

Alliance with FPT Education

Section 1: The evolution of deployment

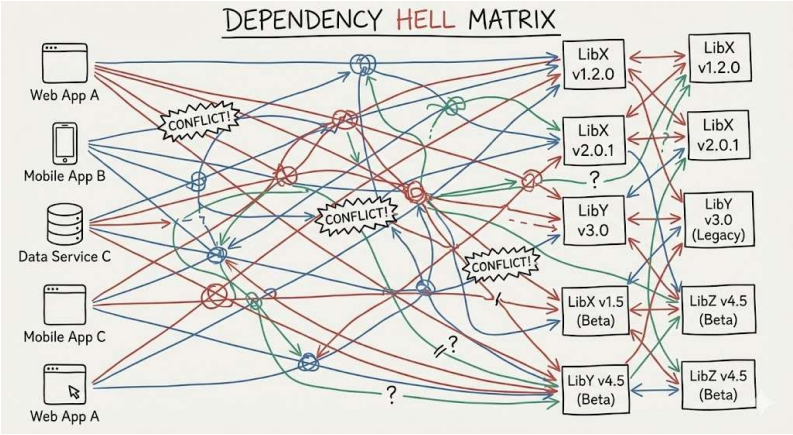
“It works on my machine” crisis

- **Scenario:**
  - You write code on Windows. It works.
  - You deploy to Linux. It crashes.
- **Why?**
  - Different OS libraries.
  - Different language versions (Ex: Node 14 vs Node 18, Python 2.7 vs Python 3.9,…)
  - Missing configuration files.
- **Result:** Developers waste 50% of their time debugging environments, not writing code.



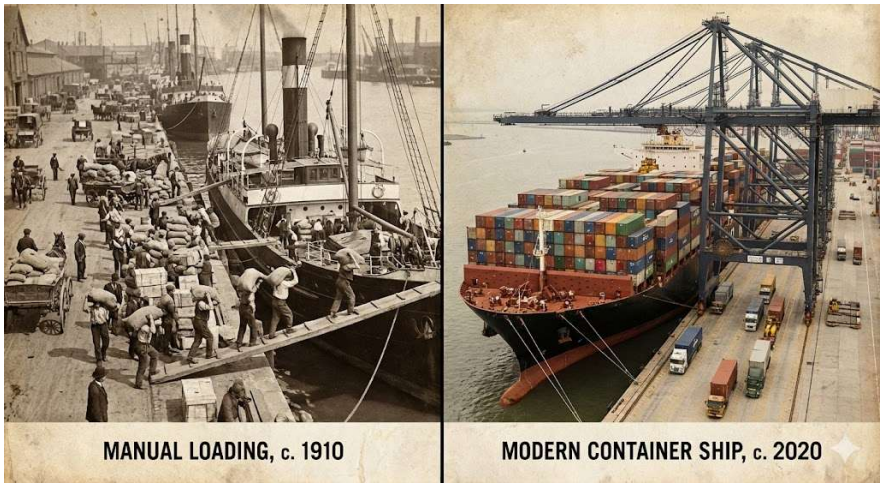
Dependency Hell

- Imagine you have 3 apps (A, B, C) and 3 environments (Dev, Test, Prod).
- App A needs Python 2.7. App B needs Python 3.9.
- Installing both on the same server creates conflicts ("Dependency Hell").
- Upgrading one app might break the others.



The Shipping Container Analogy

- **Before 1960:** Goods (Sacks, Barrels, Boxes) were loaded manually. It was slow, expensive, and fragile.
- **The Innovation:** The Standard Steel Container.
- **Impact:** The ship doesn't care what's inside (Cars, Bananas, Toys). It just handles the standard box.



Docker: The Software Container

Docker wraps everything an application needs into a standard unit:

The Code.

The Runtime (Node, Python, Java).

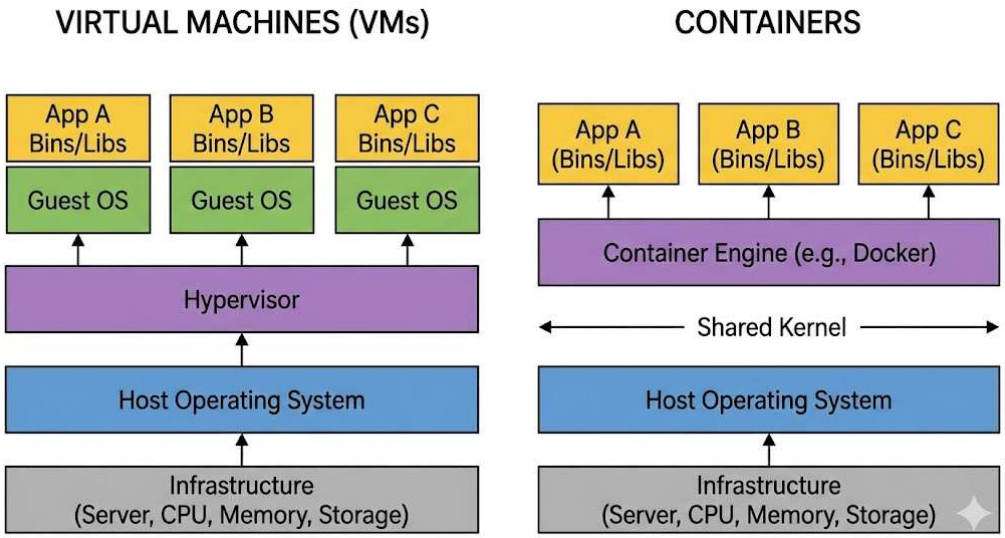
System Tools & Libraries.

Settings.

Guarantee: If it runs in Docker on your laptop, it runs in Docker on the Server.

Virtual Machines vs Containers

- **Virtual Machine (VM):** Virtualizes the **Hardware**.
  - Each VM has a full Guest OS (GBs in size).
  - Slow boot (minutes).
- **Container:** Virtualizes the **OS**.
  - Shares the Host Kernel.
  - Lightweight (MBs).
  - Instant boot (milliseconds).



Benefits of Containerization

Speed: Spin up new environments in seconds.

Efficiency: Run 10x more containers than VMs on the same hardware.

Isolation: If one container crashes, it doesn't kill the server.

Portability: "Build Once, Run Anywhere."

UNIVERSITY OF GREENWICH

Alliance with FPT Education

Section 2: Docker Core Concepts

Multi-Stage Dockerfile

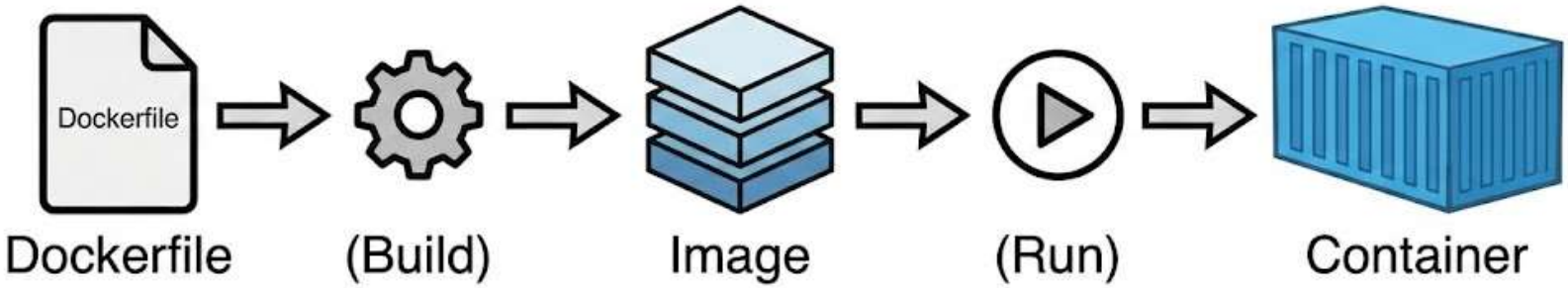
- Reduce image size and improve security by separating build and runtime environments
- This creates a smaller image by only copying the necessary files from the build stage.

```
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
# Build step...

FROM node:18-alpine AS production
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
EXPOSE 3000
CMD ["npm", "start"]
```

The Docker Trinity

- **Dockerfile:** The Recipe (Text file)
- **Image:** The Template (Built from the recipe)
- **Container:** The Instance (Running process)



Concept 1 – The Dockerfile

- A simple text file named Dockerfile (no extension).
- Contains a list of instructions to assemble the image.
- It is essentially a script that automates the installation steps.
- *We will write one in the next section.*

Concept 2 – The Docker Image

An executable package that includes everything needed to run an application.

Read-Only: Once built, an image cannot be changed (immutable).

Layered: Built in layers like a cake.

Stored in a **Registry** (like Docker Hub).

Concept 3 – The Container

A runtime instance of an image.

It's a sandboxed process on your machine.

Ephemeral: Containers are meant to be stopped, destroyed, and replaced easily.

You can run multiple containers from the same image.

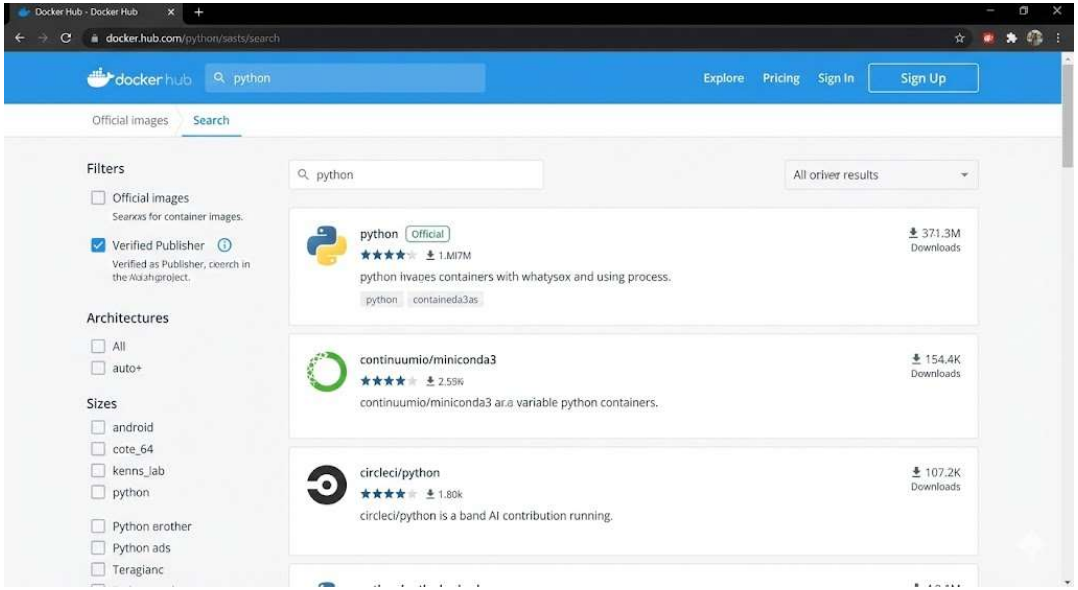
Docker vs Virtual Machines

Aspect	Docker	Virtual Machine
Size	Lightweight (MBs)	Heavy weight (GBs)
Performance	High, as containers share the host OS	Lower, as each VM run its own OS
Boot Time	Fast (seconds)	Slow (minutes)
Isolation	Good, but not as strong as VMs	Strong isolation between VMs
Portability	Very portable (works across any systems with Docker instead)	Less portable, dependent on hypervisor

Docker Hub (Registry)

The App Store for Images

- **Docker Hub:** A cloud repository to store and share images.
- **Official Images:** Verified images from vendors (Node, Python, Mongo, Nginx).
- *Always use Official Images as your base.*



Docker Hub

Image Management

- Push and pull images from Docker Hub

```
docker push my_image_name
```

```
docker pull my_image_name
```

- Tagging images is important for versioning

```
docker tag my_image_name my_repo/my_image_name:1.0
```

UNIVERSITY OF GREENWICH

Alliance with FPT Education

Section 3: Docker Deep Dive

Anatomy of a Dockerfile

Multi-Stage Build

- Building a Node.js image with Multi-Stage Dockerfile
- Let's create a proper Dockerfile for a simple Express.js app.

```
# [BUILDER STAGE]
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
# RUN npm test / npm run build (if needed)

# [PRODUCTION STAGE]
FROM node:18-alpine AS production
WORKDIR /app
COPY --from=builder /app/package*.json ./
RUN npm ci --only=production
COPY --from=builder /app ./
EXPOSE 8080
CMD ["npm", "start"]
```

- 2 FROM statements = 2 stages (builder + production)
- Builder stage: Install ALL deps (dev + prod), copy all code
- Production stage: Copy ONLY needed artifacts from builder
- Result: Smaller image! (1.2GB build → 28MB production)

FROM

Every Dockerfile must start with FROM.

It sets the **Base Image**.

Why Alpine? It is a minimal Linux distribution (~5MB). Standard Linux images are ~100MB+. Using Alpine makes your image smaller and more secure.

```
FROM node:18-alpine
```

WORKDIR

- Creates a directory **/app** inside the container.
- Sets it as the default location for subsequent commands.
- Prevents you from cluttering the root directory (/).

```
WORKDIR /app
```

COPY

Copies ***package.json*** from Host to Container.

Why only this file? Because we want to cache the dependency installation step.

```
COPY package*.json ./
```

RUN

Executes a command *during the build phase*.

This downloads libraries and saves them into the image layer.

CI = Clean Install (faster, safer for Docker)

```
RUN npm ci
```

COPY (Source Code)

Copies all your source code (**index.js, src/)** into the container.

Note: This comes **after** npm install to optimize build speed.

```
COPY . .
```

EXPOSE

- Documentation only.
- Tells the user that the application inside listens on port 8080.
- It does *not* actually publish the port to the host (you do that at runtime).

```
EXPOSE 8080
```

CMD

The command that runs when the **Container Starts**.

There can be only one CMD per **Dockerfile**.

```
CMD ["npm", "start"]
```

Optimization

Layer Caching

Docker caches every layer.

Scenario: You change one line of code in **app.js**.

Result: Docker sees **package.json** hasn't changed
⇨ Skips **npm install** (Instant!) ⇨ Re-runs **COPY** . ..

*If you copied everything at the start, **npm install** would run every single time.*

UNIVERSITY OF GREENWICH

Alliance with FPT Education

Section 4: CLI commands & Networking

Building the Image

- **build** : Create image.
- **-t** : Tag (Name) the image **my-app:v1**.
- **.** : Context. Look for Dockerfile in the *current directory*.

```
docker build -t my-app:v1 .
```

Running the Container

- **-d**: Detached. Run in background.
- **-p 8080:3000**: *Port Mapping*.
  - Forward traffic from Host Port **8080** ⇨ Container Port **3000**
  - Access app at **localhost:8080**

```
docker run -d -p 8080:3000 my-app:v1
```

Managing Containers

`docker ps` : List running containers.

`docker logs <container_id>` : View output (console.log).

`docker stop <container_id>` : Stop gracefully.

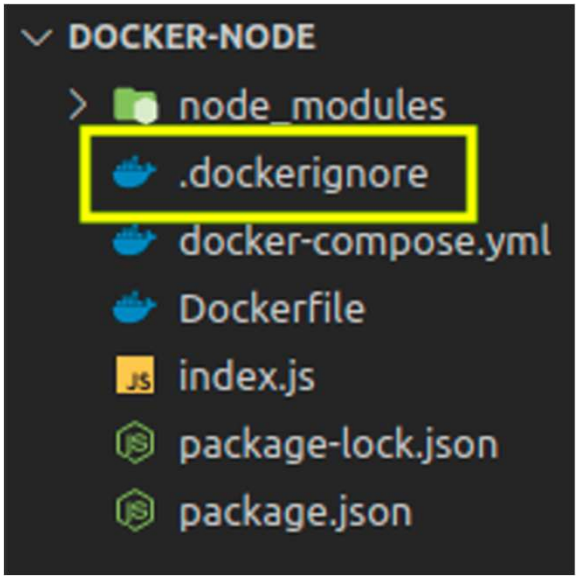
`docker rm <container_id>` : Delete the container.

`docker system prune` : Clean up unused space.

Ignoring Files

.dockerignore

- Similar to **.gitignore**
- **Crucial:** Always add **node_modules** to **.dockerignore**
- **Why?** You want dependencies installed *inside* the Linux container, not copied from your Windows/Mac host
- Also ignore: **.git**, **.env**, **Dockerfile**



Data Persistence

Volumes with PostgreSQL

- **Problem:** When a container is deleted, its file system is destroyed. Database data is LOST forever.
- **Solution: Volumes.**
  - Mount a folder on your Host Machine to a folder inside the Container
  - ⇨ Data survives container restart/delete!

```
docker run -v ./data:/var/lib/postgresql/data \
  -e POSTGRES_PASSWORD=password \
  postgres:15-alpine
```

UNIVERSITY OF GREENWICH

Alliance with FPT Education

Section 5: Docker Compose

The Multi-Service Problem

Beyond *"Hello World"*

- Real apps are not islands. They need:
  - Database (MongoDB, Postgres).
  - Cache (Redis).
  - Backend API.
- Running 3 separate docker run commands and connecting them manually is painful.

Introduction to Docker Compose

- A tool for defining and running multi-container Docker applications.
- Uses a single YAML file: **docker-compose.yml**.
- **Philosophy:** "*Infrastructure as Code*" for local development.

Docker Compose

Multi-Container Application

- Docker Compose simplifies managing multi-container applications
- Define multiple services (e.g., web, database, cache) in a single **docker-compose.yml** file
- Services communicate with each other via their service names (e.g., ‘db’, ‘web’)

Docker Compose

Multi-Service Application

- **PROBLEM**: Running app + database separately is tedious
  Docker run for app...
  Docker run for database...
  Network them together...

  → MANUAL, ERROR-PRONE!

- **SOLUTION**: Docker Compose
  One file.
  Define both services.
  One command to run both.

- 1 command for EVERYTHING:

```
docker-compose up -d
```

```yaml
version: '3.8'

services:
  # SERVICE 1: Your Node.js Application
  app:
    build: .                    # Build from local Dockerfile
    ports:
      - "8080:3000"           # Host:Container port mapping
    environment:
      - DB_HOST=db             # Service name = hostname!
      - DB_PORT=5432
      - DB_USER=postgres
      - DB_PASSWORD=postgres_password
    depends_on:
      - db                     # Wait for db to start first

  # SERVICE 2: PostgreSQL Database
  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres_password
      POSTGRES_DB: tododb
    volumes:
      - db_data:/var/lib/postgresql/data  # Persistent data

# NAMED VOLUME
volumes:
  db_data:
```

Anatomy of docker-compose.yml

The Compose File

- This configures a web app, PostgreSQL, and Redis container to communicate via service names

```yaml
version: '3'
services:
  web:
    image: my_web_app
    ports:
      - "8080:80"
    depends_on:
      - db
  db:
    image: postgres
    environment:
      POSTGRES_PASSWORD: example
  redis:
    image: redis
    ports:
      - "6379:6379"
```

Networking in Compose

Service Discovery

- **Default Network**: All services in a Compose file are connected to the same default network
- Compose automatically creates a private network.
- Containers can reach each other by **Service Name**.

❌ `mongoose.connect('mongodb://localhost:27017/myapp')` (Fail)

✅ `mongoose.connect('mongodb://db:27017/myapp')` (Success)

Docker Compose

Service Discovery & Environment Variables

- **KEY CONCEPT**: Service Name = Hostname (DNS)
- When app service needs to connect to database:
  - INCORRECT : const conn = connect('localhost:5432')
  - CORRECT : const conn = connect('db:5432')
- Docker Compose automatically configures network DNS:
  - Service "**db**" → Accessible as "**db**" to other services
  - Service "**app**" → Accessible as "**app**" to other services

```
# ENVIRONMENT VARIABLES IN COMPOSE:

depends_on:
  - db                              # Service must start first

environment:
  - DB_HOST=db                      # Not "localhost"!
  - DB_PORT=5432                    # Postgres default port
  - DB_USER=postgres                # Must match db service!
  - DB_PASSWORD=postgres_password
```

```
/* YOUR APPLICATION reads from environment: */
const dbHost = process.env.DB_HOST || 'localhost'
const dbUser = process.env.DB_USER || 'postgres'
```

```
RUN & TEST:
$ docker-compose up -d
$ docker-compose logs app       # View app logs
$ docker-compose logs db        # View db logs
$ docker-compose down           # Stop & remove
```

Docker Compose

Data Persistence

- **BIND MOUNT (OK for dev):**

  volumes:

  - ./data:/var/lib/postgresql/data

  - Stores on host disk
  - Platform-dependent paths
  - Permission issues on Windows/Mac

- **NAMED VOLUME (BEST for production):**

  volumes:

  - db_data:/var/lib/postgresql/data

  - Managed by Docker
  - Works on all platforms
  - Safer, more portable

```yaml
services:
  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_PASSWORD: password
    volumes:
      - db_data:/var/lib/postgresql/data

volumes:
  db_data:    # Declare at top level
```

Compose Commands

`docker compose up` : Builds images (if needed) and starts all containers.

`docker compose up -d` : Starts in background.

`docker compose down` : Stops and removes everything (clean slate).

UNIVERSITY OF GREENWICH

Alliance with FPT Education